Real-Time Traffic Monitoring in Rennes

Advanced Big Data Project

ARAÚJO Joao, CÁCERES Alejandro

Context & Objectives

Data Source:

Continuous data collection from the Rennes Traffic API, which provides GPS coordinates and traffic speeds every 3 minutes.

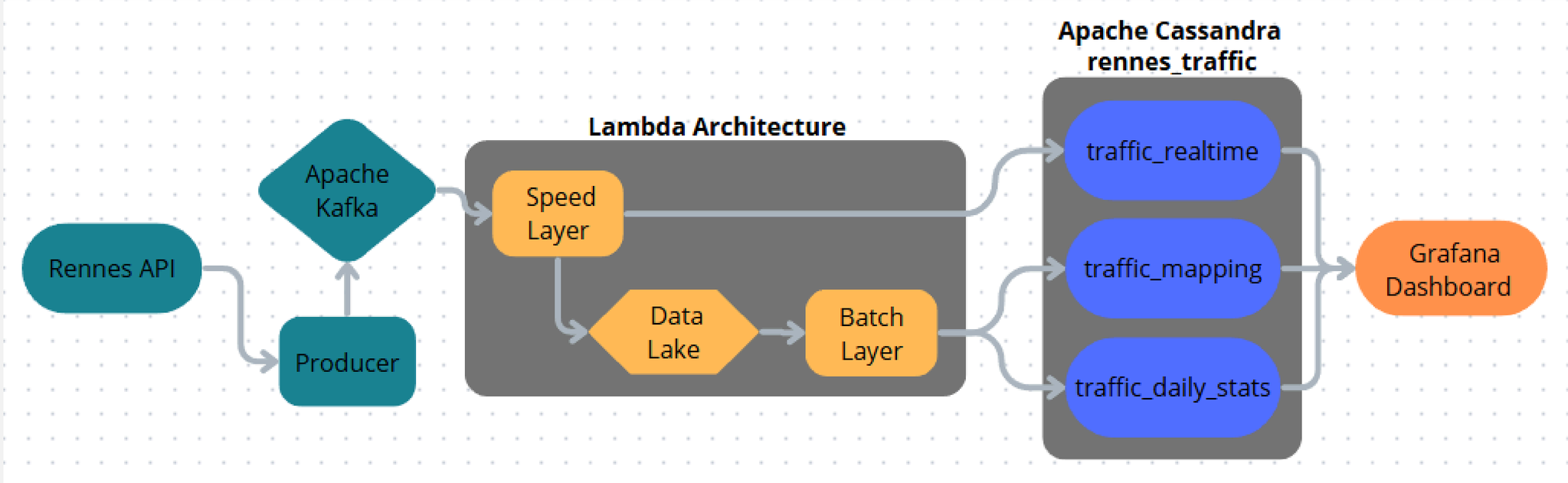
Goal 1:

Transforming raw data into simple, color-coded live maps to give drivers immediate traffic updates for their daily commutes.

Goal 2:

Processing past data to calculate daily average speeds, allowing city planners to understand long term traffic trends and improve urban infrastructure.

Lambda Architecture



Ingestion

A Python script extracts data from the API every 3 minutes and streams it into an Apache Kafka broker, ensuring a continuous and stable flow of information

The Split

The pipeline divides into two distinct paths: a Speed Layer that processes live data instantly, and a Data Lake that stores raw data for the Batch Layer

Dual Path

The Batch Layer reads historical files from the Data Lake every 9 minutes to perform heavy geographic transformations, protecting the Speed Layer from crashing

Storage

Both processing layers write their final results into a single Apache Cassandra database, which seamlessly feeds our Grafana

Speed Layer: Real Time Processing

Tool:

We use PySpark Structured Streaming to continuously consume live messages directly from our Kafka topics

Core Function:

The engine quickly filters the data to extract only the most essential metrics, such as average speed and current traffic status

Latency constraint:

To guarantee a realtime, we completely avoid heavy data transformations in this layer, ensuring the information reaches Cassandra in just seconds

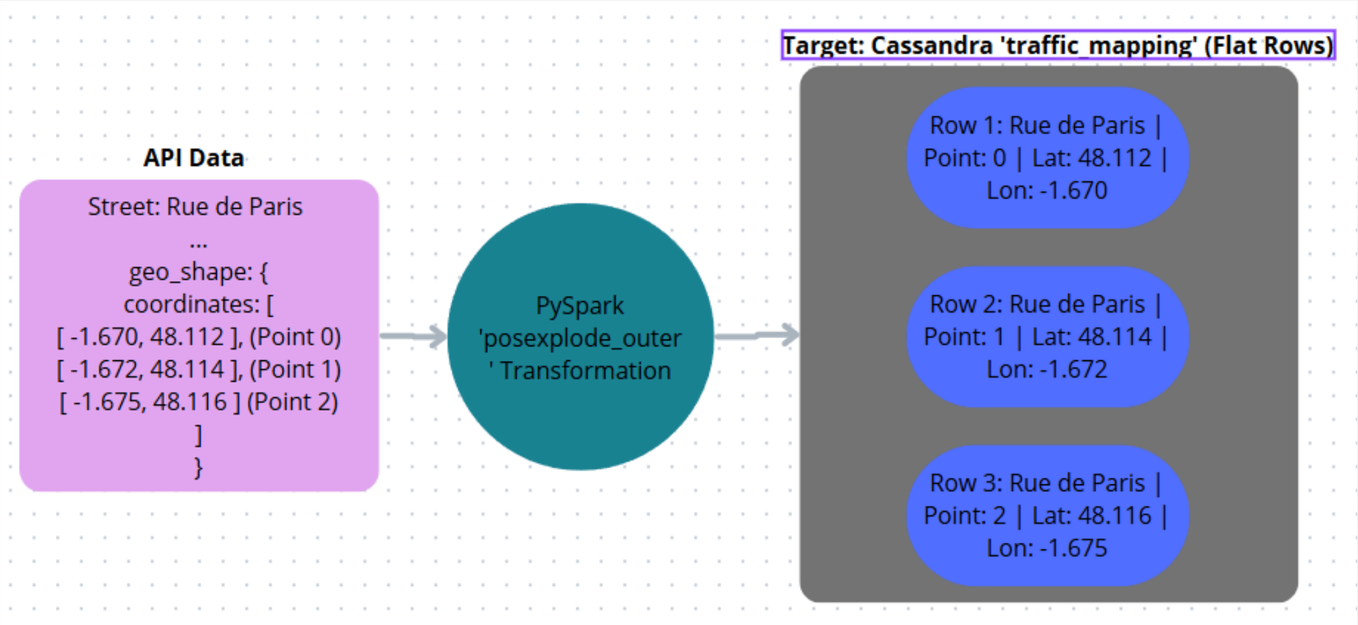
Batch Layer & The Transformation Challenge

The Problem

The API provides the maps as nested arrays of coordinates

The Solution

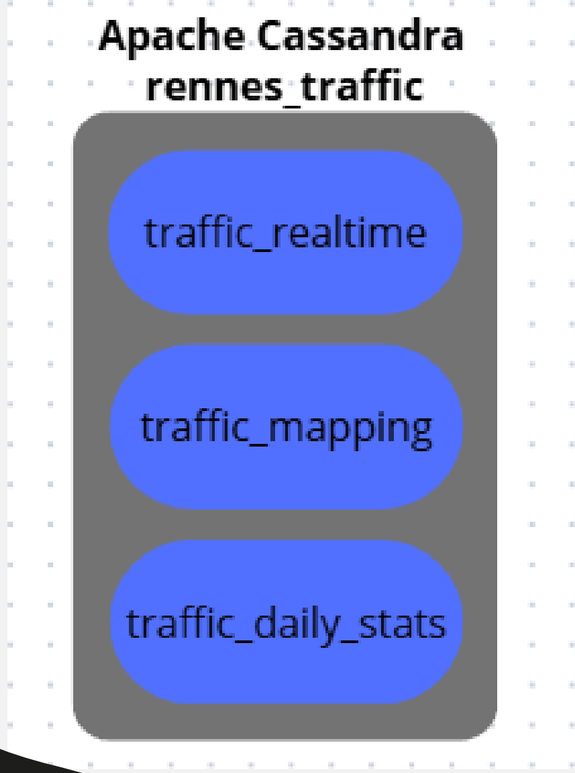
The batch layer uses a PySpark posexplode transformation to "flatten" these arrays , creating a completely new row for every individual GPS point



Why Batch?

We intentionally moved this flattening task to the Batch Layer because this multiplication process took too long and broke the continuous flow of our Speed Layer

Data Storage: Cassandra Modeling



Speed Layer Storage

The traffic_realtime table instantly receives live updates, which directly powers the current traffic conditions on our map

Batch Layer Storage

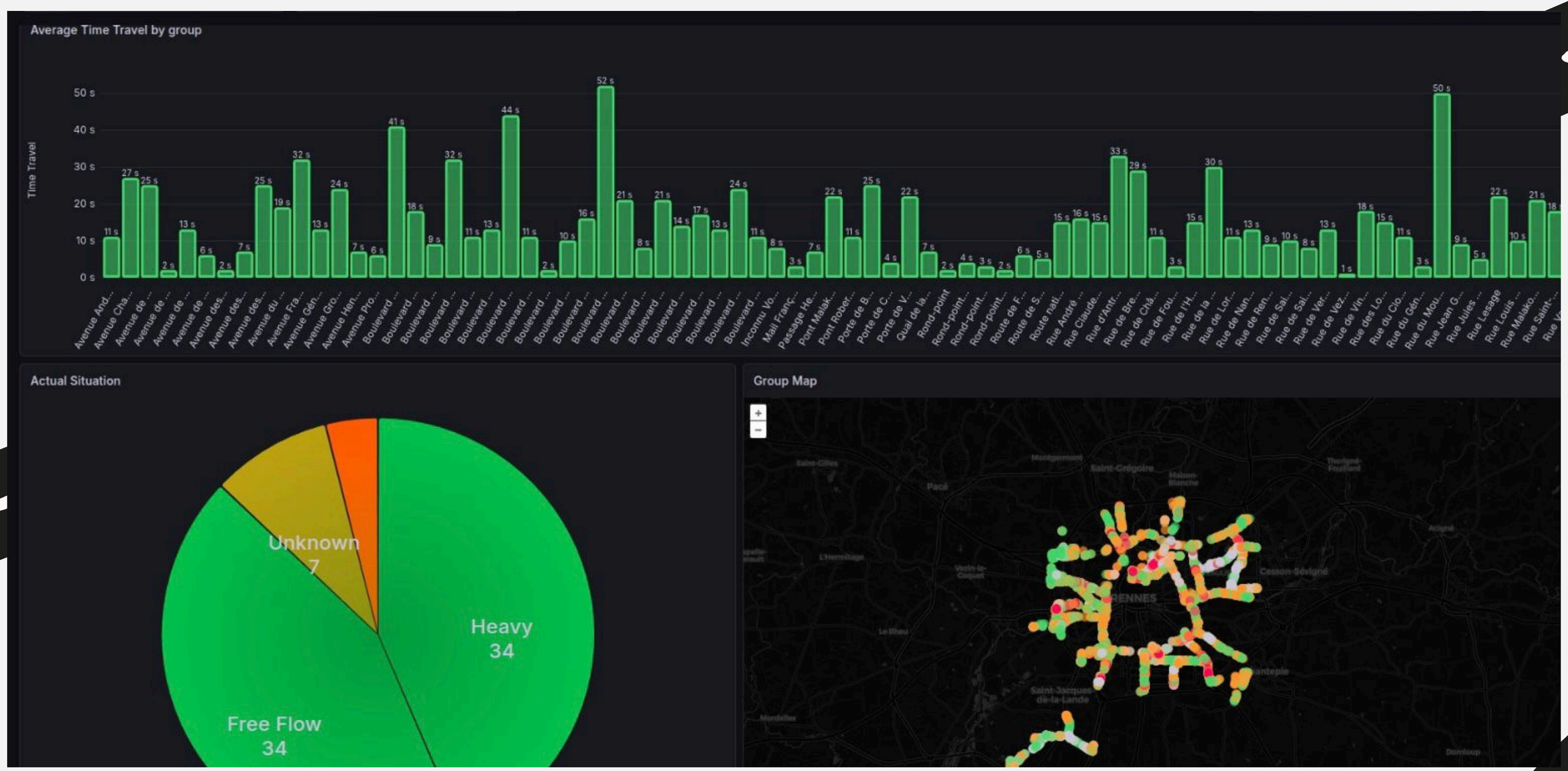
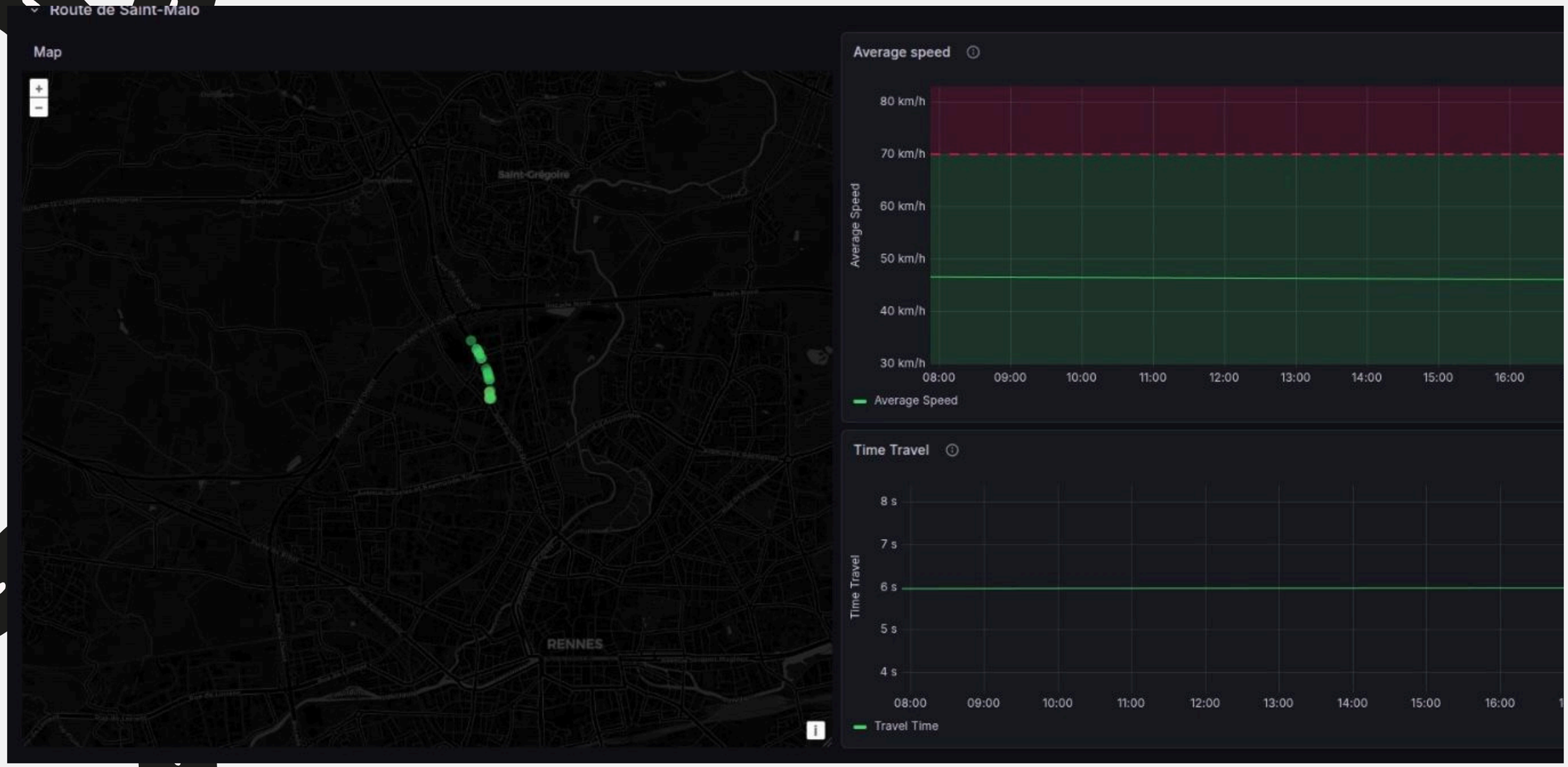
The traffic_mapping and traffic_daily_stats tables store the heavy historical data, providing the foundation for our daily average speed charts

Performance Design

We specifically used street_name as the Partition Key and date_time (descending) as the Clustering Column, which guarantees instant queries without scanning the entire database

Data Visualization

Data Visualization



Data Visualization